

API Development with Node.js, Express.js, MongoDB, and Postman

A Complete Tutorial Summary

1. Introduction to APIs

1.1 What is an API?

API stands for **Application Programming Interface**. It is an interface that allows different technologies (web apps, mobile apps, frontend frameworks, etc.) to communicate with a server and database.

1.2 Why APIs are Needed

Situations that require APIs:

- A web application built with Node.js + MongoDB needs the same data available inside Android or iOS apps.
- React, Angular, Vue, or any frontend SPA cannot directly communicate with a database.
- Native mobile technologies (Java/Kotlin for Android, Swift for iOS) also cannot connect directly to MongoDB/MySQL.

Solution: Create an API layer on the server, which handles:

- Taking requests from any client
- Reading/writing data in the database
- Sending back data in **JSON**, **XML**, or **GraphQL** format

JSON is most commonly used because it is simple, lightweight, and fast.

2. CRUD Operations in APIs

APIs commonly perform:

- **GET** → Read data
- **POST** → Create new data
- **PUT** → Update existing data
- **DELETE** → Delete data

Unlike standard server-rendered Express apps where GET and POST are often enough, **APIs must use the proper HTTP methods**.

3. Setting up the Project

3.1 Project Initialization

```
npm init -y
```

3.2 Install Required Packages

```
npm install express mongoose nodemon
```

3.3 Configure npm script

In `package.json`:

```
"start": "nodemon index.js"
```

4. Folder Structure

```
project/  
├── index.js  
├── .env  
├── package.json  
├── models/  
│   └── Student.model.js  
├── routes/  
│   └── Students.routes.js  
└── config/  
    └── database.js
```

5. Express Server Setup (`index.js`)

5.1 Basic Express Code

```
import express from "express";  
import connectDB from "../config/database.js";  
import studentRoutes from "../routes/Students.routes.js";  
  
const app = express();  
  
// Middleware for JSON body parsing
```

```
app.use(express.json());

// Load environment variables
import dotenv from "dotenv";
dotenv.config();

// Connect MongoDB
connectDB();

// API base route
app.use("/api/students", studentRoutes);

// Start server
const PORT = process.env.PORT;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

6. MongoDB Configuration (`config/database.js`)

```
import mongoose from "mongoose";
import dotenv from "dotenv";

dotenv.config();

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URL);
    console.log("Connected to MongoDB");
  } catch (err) {
    console.error(err);
  }
};

export default connectDB;
```

7. Environment Variables (`.env`)

```
PORT=3000
MONGO_URL=mongodb://127.0.0.1:27017/students_crud
```

8. Creating the Mongoose Model (`Student.model.js`)

```
import mongoose from "mongoose";

const studentSchema = new mongoose.Schema({
  first_name: { type: String, required: true },
  last_name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  phone: { type: String, required: true },
  gender: { type: String, required: true, enum: ["male", "female", "other"] },
  profile_pic: { type: String }
});

export default mongoose.model("Student", studentSchema);
```

9. Building CRUD API Routes (`Students.routes.js`)

```
import express from "express";
import Student from "../models/Student.model.js";

const router = express.Router();

// 1. GET ALL students
router.get("/", async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// 2. GET single student
router.get("/:id", async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);

    if (!student) {
      return res.status(404).json({ message: "Student not found" });
    }

    res.json(student);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
```

```
// 3. CREATE new student
router.post("/", async (req, res) => {
  try {
    const newStudent = await Student.create(req.body);
    res.status(201).json(newStudent);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});

// 4. UPDATE student
router.put("/:id", async (req, res) => {
  try {
    const updatedStudent = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );

    if (!updatedStudent) {
      return res.status(404).json({ message: "Student not found" });
    }

    res.json(updatedStudent);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});

// 5. DELETE student
router.delete("/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);

    if (!student) {
      return res.status(404).json({ message: "Student not found" });
    }

    res.json({ message: "Student deleted" });
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});

export default router;
```

10. Testing APIs Using Postman

10.1 GET All Students

```
GET /api/students
```

10.2 GET Single Student

```
GET /api/students/:id
```

10.3 POST – Create Student

Body → **form-url-encoded** Fields:

- first_name
- last_name
- email
- phone
- gender

10.4 PUT – Update Student

```
PUT /api/students/:id
```

Body fields: values you want to update.

10.5 DELETE – Remove Student

```
DELETE /api/students/:id
```

11. Transition from EJS to Twig (In API Context)

While APIs do not render HTML templates, if any server-rendered sections previously used **EJS**, replace them with **Twig** such as:

Example (for reference only)

```
<h1>{{ student.first_name }} {{ student.last_name }}</h1>
```

No rendering statements like `res.render()` are used in pure API development.

12. Summary of Learning

This tutorial covered:

- What an API is and why it is used
 - Difference between frontend frameworks and server APIs
 - HTTP methods (GET, POST, PUT, DELETE)
 - Setting up Express with JSON middleware
 - Connecting MongoDB with Mongoose
 - Creating schema, model, and CRUD routes
 - Handling errors using try/catch
 - Testing API endpoints with Postman
 - Abstracting configuration using `.env` and `/config` directory
-